

Akamata

Zig 0.16 製のモダン Web フレームワーク
VPS でも Cloudflare Workers でも、同じコードで動く

v0.3 · 2026

なぜ作ったのか

- **Zig** のパフォーマンスと安全性 (依存ゼロ、メモリ管理が明示的) を Web に持ち込みたい
- 「**VPS で動く小さな静的バイナリ**」と「**Cloudflare Workers の wasm モジュール**」を **1つのソース** から両方ビルドしたい
- Hono の DX (ハンドラ + ミドルウェア + 型安全 JSON) を Zig で実現
- SQLite / Turso / D1 を URL 一つで切り替える透過的な DB 層

結果

本番運用可能な品質・5,300 万リクエスト連続処理してメモリリーク無し・P99 88μs・検証された HTTP smuggling 防御

Hello, Akamata

Express / Hono を触ったことがあれば、ほぼ説明不要のはずです。

```
const std = @import("std");
const am  = @import("akamata");

const State = struct {};

pub fn main() !void {
    var gpa: std.heap.DebugAllocator(.{}) = .init;
    defer _ = gpa.deinit();

    var app = am.App(State).init(gpa.allocator(), .{});
    defer app.deinit();

    _ = try app.useAll(am.mw.recover(State));
    _ = try app.get("/", hello);

    try app.serve(.{ .port = 8080 });
}
```

同じソース、2つの形態

Native バイナリ

```
zig build -Doptimize=ReleaseFast
# 静的 musl バイナリ
zig build \
  -Dtarget=x86_64-linux-musl \
  -Doptimize=ReleaseFast
```

VPS · Docker · Cloudflare Containers

Cloudflare Workers

```
zig build \
  -Dbackend=workers \
  -Doptimize=ReleaseSmall
# 約 140 KB の wasm
akamata deploy --workers
```

Edge · D1 / Turso · スケーラブル

ハンドラのコードは **完全に同じ**。 `am.backend` 定数で必要に応じて分岐可能。

DB を URL 1 つで切り替え

```
var db = try am.db.open(alloc, url);
```

URL スキーム	バックエンド	実行先
<code>file:./app.db</code>	SQLite (in-process)	native のみ
<code>libsql://<db>.turso.io?authToken=...</code>	Turso (libsql HTTP)	native + Workers
<code>https://<db>.turso.io?authToken=...</code>	Turso (HTTPS alias)	native + Workers
<code>d1:DB</code>	Cloudflare D1 binding	Workers のみ

ハンドラからは同一 vtable (`am.db.Db`)。 **SQL も bind 引数も全く同じ。**

Model 層 — struct 1 つで完結

```
pub const Todo = struct {
    id: ?i64 = null,
    title: []const u8,
    priority: i32 = 3,
    completed: bool = false,
    created_at: ?i64 = null,

    pub const __schema = .{
        .table = "todos",
        .indexes = .{ .{ .{ "completed", "priority" }, .index } },
        .defaults = .{ .created_at = "unixepoch()" },
        .validates = .{
            .title = .{ am.model.rule.required, am.model.rule.max_len(200) },
            .priority = .{ am.model.rule.range(1, 3) },
        },
    };
};
```

バリデーション、デフォルト値、インデックス、DDL 生成、マイグレーションすべてここから自動派生

型安全な CRUD — 生 SQL 不要

// 読み取り

```
const one    = try Todos.find(db, arena, 42);           // ?Todo
const all    = try Todos.all(db, arena);                // []Todo
const highs  = try Todos.where(db, arena, { .priority = 1 });
```

// 書き込み

```
const created = try Todos.create(db, arena, { .title = "buy milk" });
try Todos.save(db, arena, &updated);
try Todos.delete(db, 7);
```

// 抜け道 — 任意 SQL でも T にマップ

```
const rows = try Todos.queryRaw(db, arena,
  "SELECT * FROM todos WHERE created_at > ? ORDER BY id", {cutoff});
```

// N+1 撲滅 — 1 IN クエリで親 + 全子を取得

```
const loaded = try am.model.preload.hasMany(User, "posts", owners, db, arena);
```

ハンドラはこれだけ

Before — 13 行

```
fn createTodo(c: *Ctx) !void {
    const Body = struct { title: []const u8 };
    const body = c.req.json(Body) catch {
        return c.json(.{
            .error_kind = "bad_request",
            .message = "expected {title}",
        }, 400);
    };
    const errs = try Todos.validate(body, c.arena);
    if (errs.len > 0) {
        return c.json(.{ .errors = errs }, 422);
    }
    const t = try Todos.create(c.db(), c.arena, body);
    try c.json(t, 201);
}
```

After — 3 行

```
fn createTodo(c: *Ctx) !void {
    const todo = (try c.input(Todo)) orelse return;
    const t = try Todos.create(c.db(), c.arena, todo);
    try c.json(t, 201);
}
```

- `c.input(T)` — パース + バリデーション
- 失敗時は **400 / 422 を自動応答**
- `orelse return` で早期 return

自動マイグレーション

モデルの `__schema` を起動時に DB と diff、必要な ALTER を自動生成。

```
const plan = try am.model.migrate.diff(arena, db, &all_models);
try am.model.migrate.apply(arena, db, plan);
// → CREATE TABLE / ADD COLUMN / CREATE INDEX を必要分だけ実行
```

あるいはバージョン付きファイルで

```
$ akamata migrate generate add_email
→ migrations/20260523120000_add_email.sql 生成

$ ./myapp migrate-up
→ schema_migrations を確認、未適用分のみ実行
```

どちらの方式も SQLite / Turso / D1 全てに対応。Workers では JSPI 経由で実行。

1 コマンドでデプロイ

```
$ akamata deploy --workers \  
  --config=deploy/wrangler.toml \  
  --migrate=<(. /zig-out/bin/myapp --print-schema)
```

1. `wrangler.toml` の `[[d1_databases]]` を読む
2. プレースホルダ UUID なら `wrangler d1 create` を実行、書き戻す
3. 既存 DB ならアカウントから検出して採用
4. `--migrate` で渡されたスキーマを本番 D1 に適用
5. `zig build -Dbackend=workers`
6. `wrangler deploy`

2 回目以降は冪等。CI からも安全に呼べる。

魔法の正体: JSPI

D1 の async API を Zig 側から **同期的に** 呼べる仕組み。

JS host (index.mjs)

```
d1_step: new WebAssembly.Suspending(
  async (h) => {
    const e = d1stmts.get(h);
    return await e.iter.next();
  }
);

handleFetchAsync = WebAssembly.promising(
  exports.handle_fetch);
```

Zig 側 (d1.zig)

```
extern "akamata_d1"
    fn d1_step(stmt: i32) i32;

// 同期呼び出しに見える
while ((try stmt.step()) == .row)
    try rows.append(arena, row);
```

wasm スタックを V8 が park / resume。Zig 側は変更ゼロ、Asyncify 比でコードサイズ膨張ゼロ。

信頼性 — Production audit 済み

HTTP smuggling 防御

CL + TE 同時拒否、複数 CL 拒否、不正 chunked 拒否。**902 件の adversarial fuzz 後も panic 無し。**

JWT 安全実装

alg=none / alg=RS256 偽装拒否、HS256 明示検証 (CVE-2015-2951)、時刻検証必須。

長時間負荷で安定

5 分間 53M req 処理して RSS フラット (リーク無し)、P99 ドリフト無し。

TLS 証明書検証

SAN/CN 検証、SSL_VERIFY_PEER 強制。中間者攻撃に晒さない安全な既定値。

graceful shutdown

SIGINT/SIGTERM で listener close → drain。zero-downtime 再起動を実現。

OOM safe

per-connection arena 別離、エラーは error union で型で強制。panic 発生条件を全箇所監査済み。

パフォーマンス

vs. Go / Hono

シナリオ	Akamata	Go	Hono+Bun
/hello (text)	175k	208k	165k
/echo (JSON)	181k	175k	128k
/db (SQLite)	91k	46k (40% err)	138k

単位: req/s, M2 Pro 8 worker ・ 5 分間連続

88 μ s

P99 latency (5 分連続実測)

2.5 MB

RSS (5,300 万 req 処理後)

Observability — 即運用可能

```
$ curl https://app.example.com/metrics
```

```
akamata_requests_total 5891
akamata_requests_by_status{class="2xx"} 5700
akamata_requests_by_status{class="5xx"} 3
akamata_requests_by_method{method="POST"} 1280
akamata_request_latency_seconds_bucket{le="0.0001"} 4810
akamata_request_latency_seconds_sum 0.872413
akamata_process_resident_memory_bytes 2621440
akamata_process_uptime_seconds 1234
```

- カラリティ固定で TSDB が爆発しない (path ラベル無し、method は 8 値固定)
- 構造化アクセスログ (JSON, request_id 自動付与)
- Prometheus に `scrape_interval: 15s` で繋ぐだけ

akamata CLI

プロジェクト生成 (*native + Workers* の両方)

```
$ akamata init myapp --target=both
```

開発

```
$ akamata dev # zig build run のエイリアス
```

マイグレーション

```
$ akamata migrate generate add_users
```

```
$ akamata migrate up
```

デプロイ

```
$ akamata deploy --workers --migrate=<(. /myapp --print-schema)
```

```
$ akamata deploy --containers
```

タイポにも親切

```
$ akamata deplyo
```

```
akamata: unknown subcommand `deplyo`
```

```
Did you mean `akamata deploy`?
```

標準ミドルウェアが豊富

recover

panic を 500 にして握る。一番外側に置く

logger / accessLog

stdout / JSON 構造化アクセスログ

requestId

X-Request-ID 自動生成・伝播

metrics

Prometheus テキスト形式エクスポート

jwt / bearerAuth

HS256/RS256、CVE-2015-2951 対策済み

cors

プリフライト + Origin allowlist

rateLimit

token bucket per IP / per key

session / csrf

署名付き Cookie + Double Submit

serveStatic

ファイル配信 (ETag, mime 推定)

WebSocket — HTTP ルートから upgrade

```
_ = try app.ws("/rooms/:id/ws", wsRoom);

fn wsRoom(c: *Ctx) !void {
  const room_id = try c.req.paramAs(u64, "id");
  var conn = try am.ws.upgrade(Ctx, c, {.max_message_bytes = 64 * 1024 });
  try c.state().hub.attach(room_id, &conn);
  defer c.state().hub.detach(room_id, &conn);

  while (true) {
    const msg = conn.readMessage() catch return;
    defer msg.deinit();
    if (msg.opcode == .text)
      try c.state().hub.broadcast(room_id, msg.payload);
  }
}
```

Workers では Durable Object に自動委譲。ハンドラのコードは変えずに済む。

バリデーション — 宣言 1 行で完結

```
pub const __schema = .{
  .validates = .{
    .email = .{
      am.model.rule.required,
      am.model.rule.max_len(255),
      am.model.rule.format(.email),
    },
    .age   = .{ am.model.rule.range(0, 150) },
    .name  = .{ am.model.rule.required, am.model.rule.custom(noScriptTag) },
  },
};

fn noScriptTag(value: []const u8, _: std.mem.Allocator) ?[]const u8 {
  if (std.mem.indexOf(u8, value, "<script>") != null)
    return "must not contain <script>";
  return null;
}
```

標準ルール: `required`, `min_len`, `max_len`, `range`, `format(.email/.url/.alphanumeric)`, `custom`, `customInt`

Eager loading — N+1 撲滅

```
// User の has_many :posts 関係
const users = try Users.all(db, arena);           // 1 クエリ
const loaded = try am.model.preload.hasMany(      // 1 IN クエリ
  User, "posts", users, db, arena);

for (loaded) |row| {
  // row.parent: User
  // row.related: []Post ← 既にロード済み
  std.log.info("{s} has {d} posts", .{
    row.parent.name, row.related.len
  });
}
```

N 個の親 → **合計 2 クエリ**だけ。 `WHERE user_id IN (?, ?, ?, ...)` でバッチ取得。

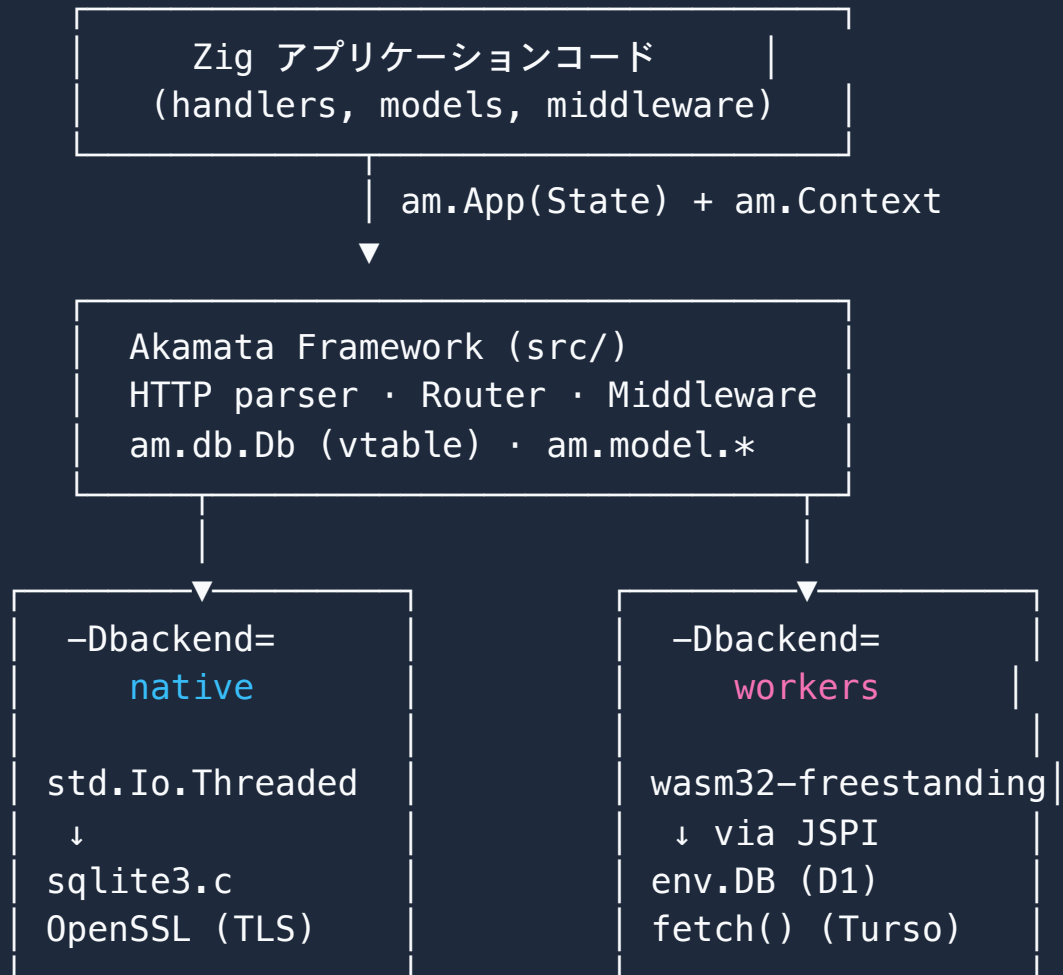
HTML UI も同梱 — 静的アセット不要

```
const index_html = @embedFile("index.html");

fn index(c: *Ctx) !void {
    const accept = c.req.header("accept") orelse "";
    if (std.mem.indexOf(u8, accept, "text/html") != null) {
        return c.html(index_html); // ブラウザ → HTML UI
    }
    try c.json(.{ .endpoints = .{ ... } }, 200); // curl → JSON
}
```

- `@embedFile` でコンパイル時にバイナリ/wasm に焼き込み
- 同じバイナリだけで配信、追加デプロイ不要
- Workers 上でも **CDN なしで HTML 配信**

アーキテクチャ全体像



どんなアプリ向けか

サーバレス API

Workers + D1 でスケール。コールドスタート ~30ms、wasm 約 150KB。

モバイル/SaaS バックエンド

JWT 認証、フレンド機能、メッセージング、push 通知の標準装備。

リアルタイム

WebSocket ハブ、Durable Object 対応、room/user 単位の broadcast。

マイクロサービス

Container 配信 1.8MB 静的 binary、起動 ~10ms、metrics 標準装備。

セキュリティ重視

production audit 済み、HTTP smuggling/JWT 偽装/MITM 防御。

CRUD/管理画面

Model + auto-migrate + validation + HTML embed で 1 ファイル完結。

競合との比較

	Akamata	Hono (Bun/Node)	Go net/http	Express
言語	Zig	TypeScript	Go	JavaScript
VPS デプロイ	✓ 静的 musl	⚠ ランタイム必要	✓ 静的	⚠ Node 必要
Workers	✓ wasm	✓ ネイティブ	✗	✗
D1 同期呼び出し	✓ JSPI	N/A (async)	—	—
SQLite 同梱	✓	via bun:sqlite	外部	外部
P99 (echo)	87 μs	~250 μs	~90 μs	~500 μs
バイナリサイズ (native)	1.8 MB	~80 MB (bun)	~10 MB	node + npm

ロードマップ

✓ v0.3 (現在)

- Model 層 + Repo + 自動マイグレーション
- Eager loading / リレーション / バリデーション
- JSPI による完全な D1 サポート
- akamata CLI 一発デプロイ
- 本番監査 (信頼性 / observability)
- 詳細チュートリアル + PDF

🚧 計画中

- OpenTelemetry エクスポート
- Reactor + io_uring / kqueue でさらなる高速化
- Enum ↔ TEXT カラム自動マッピング
- GraphQL 統合 (実験的)
- ホットリロード (開発時)
- WebTransport / HTTP/3

始めるには

```
git clone https://github.com/yourorg/Akamata
cd Akamata && zig build cli
akamata init myapp --target=both
cd myapp && zig build run
```

 **詳細チュートリアル:** `docs/tutorial.ja.md` (60–90 分で Todo アプリを完成)

 **ハンドブック:** `docs/handbook.ja.md` (15 分版)

 **サンプル:** `examples/{guestbook,chat,mobus}`

Akamata で楽しい開発を 